

Documentación de Métodos — Simulador de Gestor de Procesos

Este documento enumera y explica cada método público y privado relevante del proyecto, indicando firma, parámetros, valor devuelto, efectos secundarios y excepciones.

src/core/process.py

PCB.__post_init__(self) -> None

- Firma: def __post_init__(self) -> None
- Parámetros: self (instancia de PCB).
- Retorno: None.
- Efectos secundarios:
 - Asigna self.pid usando el contador estático PCB._next_pid.
 - Incrementa PCB._next_pid.
 - Genera valores aleatorios para cpu_burst, mem_mb y arrival_time si vienen con -1.
 - Inicializa self.time_remaining = self.cpu_burst.
- Excepciones: ValueError si cpu_burst < 1, mem_mb < 1 o arrival_time < 0.
- Uso: llamado automáticamente al crear el dataclass PCB.

PCB.transition(self, new_state: ProcessState) -> None

- Firma: def transition(self, new_state: ProcessState) -> None
- Parámetros: new_state — valor de ProcessState al que se quiere cambiar.
- Retorno: None.
- Efectos secundarios: actualiza self.state si la transición es válida.
- Excepciones: ValueError si la transición no está permitida por VALID_TRANSITIONS.
- Notas: Mantiene la integridad del ciclo de vida del proceso.

PCB.__str__(self) -> str

- Firma: def __str__(self) -> str
- Parámetros: ninguno adicional.
- Retorno: representación textual completa del PCB.
- Efectos secundarios: ninguno.
- Uso: logging y visualización en UI.

PCB.reset_pid_counter(cls) -> None (classmethod)

- Firma: @classmethod def reset_pid_counter(cls) -> None
 - Parámetros: cls.
 - Retorno: None.
 - Efectos secundarios: establece PCB._next_pid = 1.
 - Uso: reiniciar la numeración entre ejecuciones.
-

src/core/resource.py

ResourcePool.__init__(self, cpu_cores: int = 1, ram_mb: int = 4096) -> None

- Firma y parámetros: cpu_cores y ram_mb.
- Retorno: None.
- Efectos: inicializa total_cpu, total_ram, available_cpu y available_ram.
- Excepciones: ValueError si alguno de los parámetros < 1.

ResourcePool.request(self, cpu: int, ram: int) -> bool

- Firma: request(cpu, ram) -> bool.
- Parámetros: cpu (núcleos a reservar), ram (MB a reservar).
- Retorno: True si la reserva se puede efectuar; False si no hay recursos disponibles o la petición excede totales.
- Efectos secundarios: decrementa available_cpu y available_ram en caso de éxito.
- Excepciones: ValueError si cpu o ram son negativos.

- Uso: llamado por `Scheduler.admit_process` y antes de dispatch de CPU para asegurar disponibilidad.

`ResourcePool.release(self, cpu: int, ram: int) -> bool`

- Firma: `release(cpu, ram) -> bool`.
- Parámetros: `cpu, ram`.
- Retorno: `True` si la liberación fue válida; `False` si causaría sobrepaso del total (`double-free`).
- Efectos secundarios: incrementa `available_cpu/available_ram`.
- Excepciones: `ValueError` si argumentos negativos.
- Uso: liberar CPU al pasar a `WAITING` y liberar CPU+RAM al terminar un proceso.

`src/core/scheduler.py`

Notas: los métodos que empiezan con `_` son internos, empero están documentados porque definen la lógica del planificador.

`Scheduler.register_process(self, pcb: PCB) -> None`

- Firma: `register_process(pcb)`.
- Parámetros: `pcb` — instancia `PCB` en estado `NEW`.
- Retorno: `None`.
- Efectos: añade `pcb` a `self.all_processes` y llama a `logger.log(...)` con la información de registro.
- Excepciones: ninguna explícita; espera un objeto `PCB` válido.

`Scheduler.admit_process(self, pcb: PCB) -> bool`

- Firma: `admit_process(pcb) -> bool`.
- Parámetros: `pcb` (debe estar en `NEW`).
- Retorno: `True` si la admisión (reserva RAM + cambio a `READY`) fue exitosa; `False` en caso contrario.
- Efectos:
 - Pide RAM con `resources.request(cpu=0, ram=pcb.mem_mb)`.
 - Si éxito: hace `pcb.transition(ProcessState.READY)` y `self.ready_queue.append(pcb)`.
 - Registra evento en `logger`.
- Excepciones: no lanza, devuelve `False` y registra en `logger` si el estado es incorrecto o falta RAM.

`Scheduler._handle_arrivals(self) -> None`

- Firma: `_handle_arrivals()`.
- Parámetros: ninguno.
- Retorno: `None`.
- Efectos: busca procesos `NEW` con `arrival_time <= self.clock`, ordena por `arrival_time` y los admite llamando a `admit_process`.
- Uso: llamado cada tick desde `execute_cycle`.

`Scheduler._handle_waiting(self) -> None`

- Firma: `_handle_waiting()`.
- Parámetros: ninguno.
- Retorno: `None`.
- Efectos:
 - Decrementa `pcb.io_wait_time` para cada proceso en `waiting_queue`.
 - Cuando `io_wait_time <= 0`, mueve el proceso a `READY` mediante `pcb.transition(ProcessState.READY)` y `self.ready_queue.append(pcb)`.
 - Logea la finalización de I/O.

`Scheduler._select_next(self) -> PCB | None`

- Firma: `_select_next() -> PCB | None`.
- Retorno: siguiente `PCB` a ejecutar o `None` si la cola está vacía.
- Lógica:
 - FCFS: `return self.ready_queue.popleft()`.
 - SJF: busca el `PCB` con menor `time_remaining` en la `ready_queue`, lo remueve y lo retorna.
- Efectos: modifica `ready_queue`.

Scheduler._execute_tick(self, pcb: PCB) -> bool

- Firma: `_execute_tick(pcb)` -> bool.
- Parámetros: `pcb` — proceso que está en CPU.
- Retorno: True si el proceso solicitó I/O y fue movido a WAITING; False en caso contrario.
- Efectos:
 - `pcb.time_remaining -= 1`.
 - Loggea el tick.
 - Con probabilidad `IO_PROBABILITY` (si `pcb.time_remaining > 0`) genera `io_ticks = random.randint(IO_WAIT_MIN, IO_WAIT_MAX)`.
 - Asigna `pcb.io_wait_time = io_ticks`.
 - `pcb.transition(ProcessState.WAITING)`.
 - `self.resources.release(cpu=1, ram=0)` — libera la CPU pero mantiene la RAM asignada.
 - `self.waiting_queue.append(pcb)` y `self.current_process = None`.
 - No hay excepción explícita; asume `pcb` válido y `self.resources` capaz de `release`.

Scheduler.execute_cycle(self) -> str

- Firma: `execute_cycle()` -> str.
- Parámetros: ninguno.
- Retorno: mensaje de estado legible (por ejemplo, “[Tick X] PID Y TERMINADO.” o información de CPU ociosa).
- Efectos (secuencia principal):
 1. `self.clock += 1`.
 2. `_handle_arrivals()` y `_handle_waiting()`.
 3. Si `self.current_process` is None:
 - `next_pcb = self._select_next()`.
 - Si `next_pcb` is None: loggea “IDLE” y retorna.
 - Intenta `self.resources.request(cpu=1, ram=0)`; si falla, reencola `next_pcb` y retorna.
 - `next_pcb.transition(ProcessState.RUNNING)` y `self.current_process = next_pcb`.
 4. Llama `_execute_tick(self.current_process)`.
 5. Si `_execute_tick` devuelve True, retorna mensaje indicando I/O.
 6. Si `self.current_process.time_remaining <= 0`: transita a TERMINATED, `exit_reason = ExitReason.NORMAL`, `self.resources.release(cpu=1, ram=finished.mem_mb)`, loggea, limpia `current_process`.
- Excepciones: no lanza; todas las fallas se registran y la función devuelve mensajes.

Scheduler.is_simulation_complete(self) -> bool

- Firma: `is_simulation_complete()` -> bool.
- Retorno: True si `self.all_processes` está vacío o todos los procesos están en TERMINATED.
- Uso: para detectar finalización desde la GUI.

Scheduler.set_algorithm(self, algorithm: SchedulingAlgorithm) -> None

- Firma: `set_algorithm(algorithm)`.
 - Parámetros: `algorithm` enum.
 - Efectos: actualiza `self.algorithm` y loggea el cambio.
-

src/ipc/producer_consumer.py

run_demo() -> None

- Firma: `def run_demo()` -> None.
 - Parámetros: ninguno.
 - Retorno: None.
 - Efectos:
 - Crea buffer, mutex (Lock), semáforos empty/full y contadores compartidos.
 - Define producer y consumer internos que producen/consumen un número fijo de items y los imprimen.
 - Lanza 2 hilos productores y 2 consumidores, hace `start()` y `join(timeout=30)`.
 - Excepciones: la función atrapa excepciones internas solo en la integración con GUI; en modo directo imprime trazas.
-

src/ui/logger.py

Logger.log(self, message: str, tick: int = 0) -> None

- Firma: log(message, tick=0).
- Parámetros: message y tick.
- Retorno: None.
- Efectos: crea LogEntry con timestamp=time.time() y lo añade a self.entries; si self.verbose imprime la entrada en consola.

Logger.show_history(self, last_n: int = 0) -> None

- Firma: show_history(last_n=0).
- Parámetros: last_n — si > 0 filtra últimas N entradas.
- Efectos: imprime el historial formateado en consola.

Logger.clear(self) -> None

- Efectos: elimina todas las entradas de self.entries.
-

main.py — métodos de SimulatorGUI

Nota: muchos métodos de GUI son handlers; se describen aquí con su efecto en la simulación.

SimulatorGUI.on_start_simulation(self) -> None

- Firma: on_start_simulation().
- Efectos principales:
 - Si self._simulation_running (ya corriendo): pausa la simulación cancelando root.after y actualiza textos/estados.
 - Si no hay procesos en scheduler.all_processes, lee num_processes de entry_num_processes, valida rango, configura algoritmo con scheduler.set_algorithm(...) y crea PCB en NEW llamando scheduler.register_process(pcb).
 - Bloquea inputs (entry/combo) y cambia botones a estado “Pausar”.
 - Establece self._simulation_running = True y programa self._after_id = self.root.after(self.speed_var.get(), self._auto_tick).
- Excepciones: muestra messagebox.showerror si el input no es válido.

SimulatorGUI._auto_tick(self) -> None

- Firma: _auto_tick().
- Efectos:
 - Si no self._simulation_running retorna.
 - Llama self.scheduler.execute_cycle() y self.update_ui().
 - Si scheduler.is_simulation_complete() pone la interfaz en estado completado y muestra messagebox.showinfo con resumen.
 - Si no completado reprograma root.after(self.speed_var.get(), self._auto_tick).

SimulatorGUI.on_reset_simulation(self) -> None

- Efectos:
 - Cancela root.after si existe.
 - Llama PCB.reset_pid_counter().
 - Reinstancia Logger, ResourcePool y Scheduler.
 - Restaura estados de botones y entradas.

SimulatorGUI.on_view_logs(self) -> None

- Efectos: abre Toplevel, crea un Text, inserta todas las logger.entries y lo marca como state=DISABLED.

SimulatorGUI.on_run_ipc(self) -> None

- Efectos:
 - Crea Toplevel con Text y una queue.Queue de mensajes.
 - Define RedirectText que implementa write() para poner strings en la queue.
 - poll_queue() lee la queue y actualiza el Text periódicamente con top.after(50,

- `poll_queue).`
 - `run_demo_thread()` redirige temporalmente `sys.stdout` a `RedirectText` y llama `ipc_demo()`.
 - Lanza `threading.Thread(target=run_demo_thread, daemon=True).start()`.
- Notas: patrón seguro para integrar salida de hilos en la GUI.

SimulatorGUI.update_ui(self) -> None

- Efectos:
 - Actualiza `lbl_clock` con `scheduler.clock`.
 - Muestra recursos con `scheduler.resources.available_*`.
 - Calcula contadores por estado (NEW, READY, WAITING, TERMINATED).
 - Rellena `listbox_ready`, `listbox_waiting`, `listbox_terminated` con representaciones de PCB.
 - Actualiza progreso y barras.

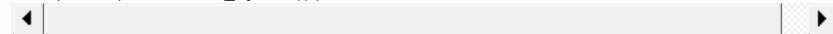
Ejemplos de uso rápido (línea de comandos)

- Registrar procesos y ejecutar un ciclo manualmente:

```
from src.core.process import PCB
from src.core.resource import ResourcePool
from src.ui.logger import Logger
from src.core.scheduler import Scheduler, SchedulingAlgorithm

res = ResourcePool(1, 4096)
log = Logger(verbose=True)
s = Scheduler(resources=res, logger=log, algorithm=SchedulingAlgorithm.FCF

p = PCB(name='Prueba', cpu_burst=5, mem_mb=128, arrival_time=0)
s.register_process(p)
# Simular varios ticks
for _ in range(10):
    print(s.execute_cycle())
```



Archivo generado automáticamente por el asistente.